

Spring Boot Configuration: `application.properties`, `application.yml` & Profiles

1. `application.properties`

1. Definition

- **`application.properties`** is Spring Boot's default configuration file.
 - It stores application settings like **port, database URL, logging, security, etc.**
 - Spring Boot automatically loads it during startup.
-

2. Why is it used?

- Central place to manage application configuration.
 - Avoids hardcoding values inside Java code.
-

3. Key Concepts

Configuration Properties

Store application settings like:

- Server Port
- Database URL
- Username & Password
- Logging Level

Example:

```
server.port=8080  
spring.application.name=JournalApp
```

Automatic Loading

Spring Boot automatically searches for:

```
src/main/resources/application.properties
```

No manual code is required.

Property Override Order (High Interview Question)

Highest priority wins.

Priority	Source
Command Line Arguments	Highest
Environment Variables	
application-{profile}.properties	
application.properties	Lowest

4. Syntax

```
server.port=8080

spring.application.name=JournalApp

spring.data.mongodb.uri=mongodb://localhost:27017

logging.level.root=INFO
```

5. Example

Simple Example

```
server.port=9090
```

Input

Run application

Output

```
Application starts on localhost:9090
```

Explanation

Spring Boot changes the server port automatically.

6. Important Rules

- File must be inside:

```
src/main/resources
```

- Use **key=value**
 - Duplicate keys → Last value wins.
 - Spring loads it automatically.
-

7. Common Mistakes

- Keeping file outside resources folder.
- Misspelling property names.
- Writing:

```
server.port : 8080
```

instead of

```
server.port=8080
```

8. Interview Questions

Q1. Why do we use `application.properties`?

Stores external configuration separately from Java code.

Q2. Does Spring Boot automatically load it?

Yes.

Q3. Can properties be overridden?

Yes.

Priority:

Command Line > Environment Variables > Profile > application.properties

9. Comparison

Hardcoded Values	application.properties
Inside Java Code	Outside Java Code
Difficult to change	Easy to change
Requires recompiling	No recompilation

10. Revision Box

- Default configuration file
 - Loaded automatically
 - Located in `resources`
 - Uses `key=value`
 - Stores DB, Port, Logging etc.
 - Externalizes configuration
 - Can be overridden
 - No code changes required
-

2. application.yml (YAML)

1. Definition

- YAML is another configuration format supported by Spring Boot.
 - It stores the same properties using **indentation** instead of dots.
-

2. Why is it used?

- Cleaner for large projects.
 - Easier to read nested configurations.
-

3. Key Concepts

Hierarchical Structure

Instead of

```
spring.data.mongodb.uri=...
```

YAML groups properties.

Indentation Matters

YAML uses spaces.

Wrong indentation causes errors.

4. Syntax

```
server:
  port: 8080

spring:
  application:
    name: JournalApp

data:
  mongodb:
    uri: mongodb://localhost:27017
    database: journaldb
```

5. Example

```
server:  
  port: 9090
```

Output

```
Application runs on port 9090
```

Explanation

Exactly same as properties.

6. Important Rules

- Never use Tabs.
 - Use Spaces.
 - Indentation defines hierarchy.
 - Keys end with `:`.
-

7. Common Mistakes



```
server:  
port:8080
```



```
server:  
  port: 8080
```

8. Interview Questions

Q1. Difference between properties and YAML?

Only syntax differs.

Functionality is same.

Q2. Which one is better?

YAML is better for complex nested configuration.

Properties is simpler.

9. Comparison

application.properties	application.yml
key=value	indentation
Flat	Hierarchical
Easier for beginners	Easier for large projects

10. Revision Box

- Alternative configuration format
 - Uses indentation
 - Same functionality as properties
 - Better readability
 - No tabs
 - Supports nested configuration
-

3. Spring Boot Profiles

1. Definition

- **Profiles** allow different configurations for different environments.
 - One application can behave differently in **Development, Testing, Staging, and Production**.
-

2. Why is it used?

- Avoid editing configuration before deployment.
 - Same code, different settings.
-

3. Key Concepts

Environment

Common environments:

Development

Testing

Staging

Production

Profile Files

application.properties

application-dev.properties

application-test.properties

application-prod.properties

or

application.yml

application-dev.yml

application-prod.yml

Active Profile

Spring loads configuration based on

spring.profiles.active=**dev**

Configuration Loading

Suppose

```
application.properties  
  
↓  
  
server.servlet.context-path=/api
```

and

```
application-dev.properties  
  
↓  
  
server.port=8080
```

Spring loads both.

Common settings come from

```
application.properties
```

Profile-specific settings come from

```
application-dev.properties
```

Command Line Override (High Interview)

```
java -jar app.jar --spring.profiles.active=prod
```

Runs application using production profile.

Environment Variables

Production servers usually activate profiles using

```
SPRING_PROFILES_ACTIVE=prod
```

instead of modifying files.

4. Syntax

application.properties

```
spring.profiles.active=dev
```

Dev Profile

```
server.port=8080  
  
logging.level.root=DEBUG
```

Production Profile

```
server.port=80  
  
logging.level.root=ERROR
```

5. Example

Simple Example

```
application-dev.properties  
  
↓  
  
server.port=8080
```

```
application-prod.properties  
  
↓
```

```
server.port=80
```

Run

```
spring.profiles.active=prod
```

Output

```
Application starts on Port 80
```

Explanation

Production configuration gets loaded.

Interview Example

Run same JAR on different ports.

Development

```
java -jar app.jar --spring.profiles.active=dev
```

Production

```
java -jar app.jar --spring.profiles.active=prod
```

Explanation

No code changes required.

6. Important Rules

- Only one profile is active unless multiple are specified.
 - Profile configuration overrides common configuration.
 - Keep common properties in `application.properties`.
-

7. Common Mistakes

- Forgetting to activate profile.
 - Putting common properties inside every profile.
 - Editing production configuration manually.
-

8. Interview Questions

Q1. Why do we use Profiles?

Different configurations for different environments.

Q2. What is the default profile?

No profile.

Only `application.properties` is loaded.

Q3. Can multiple profiles be active?

Yes.

Example

```
spring.profiles.active=dev,test
```

Q4. Which configuration has higher priority?

```
application-prod.properties
```

```
>
```

```
application.properties
```

Q5. How do companies use Profiles?

- Local → Dev Profile

- QA → Test Profile
- Client Demo → Stage Profile
- Live Server → Production Profile

Same code.

Different configuration.

9. Comparison

Without Profiles	With Profiles
Edit file before deployment	No editing
Risk of mistakes	Safe
One configuration	Multiple environments
Manual changes	Automatic

10. Revision Box

- Profiles = Multiple configurations
- Dev, Test, Stage, Prod
- Activate using `spring.profiles.active`
- Same code, different settings
- Common config → application.properties
- Environment config → application-{profile}.properties
- Production never edits Java code
- Can activate through IDE, JAR, Environment Variable
- Command line overrides properties

★ Placement Interview Summary

Frequently Asked Questions

Question	Answer
Difference between <code>.properties</code> and <code>.yml</code> ?	Only syntax; functionality is the same.
Why Profiles?	Different configurations for different environments.
Where is configuration stored?	<code>src/main/resources</code>

Question	Answer
How to activate a profile?	<code>spring.profiles.active=dev</code> or <code>--spring.profiles.active=dev</code>
Which has higher priority?	Command Line > Environment Variables > Profile Files > application.properties
Why avoid hardcoding?	Easier maintenance and deployment without changing code.
Is this used in real companies?	Yes. Almost every Spring Boot project uses external configuration and profiles.